

# Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

# Sumário

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Fundamentos de Orientação a Objetos</b>    | <b>3</b> |
| 1.1      | Abstração . . . . .                           | 3        |
| 1.2      | Objeto . . . . .                              | 4        |
| 1.2.1    | Estado de um objeto e ciclo de vida . . . . . | 4        |
| 1.2.2    | Identidade e igualdade de um objeto . . . . . | 4        |
| 1.3      | Classe . . . . .                              | 5        |
|          | <b>Referências Bibliográficas</b>             | <b>8</b> |

# 1 Fundamentos de Orientação a Objetos

Após a discussão sobre as diferenças entre os paradigmas de programação, este capítulo aprofunda-se nos conceitos fundamentais da Programação Orientada a Objetos (POO), apresentando definições e exemplos práticos. Paralelamente, serão introduzidos os diagramas de classes da (Linguagem de Modelagem Unificada) UML, que servirão de suporte para a modelagem dos sistemas abordados ao longo do material.

## 1.1 Abstração

O próximo conceito parte de algo do nosso cotidiano. A abstração nada mais é que uma forma de generalizar as características de um objeto. A abstração pode estar ligada diretamente ao que se deseja atingir, logo o problema que se deseja resolver pode influenciar em diferentes níveis de abstração.

Vamos a um exemplo: uma clínica veterinária deseja adicionar ao seu sistema uma forma de cadastrar informações no seu sistema, inicialmente ela deseja registrar os pets. De acordo com a definição de abstração, a empresa deve definir os animais da maneira mais simples possível. Portanto, será possível criar variações de animais como cachorro, gato, passaros entre outros. Observe a Figura 1.1 abaixo.

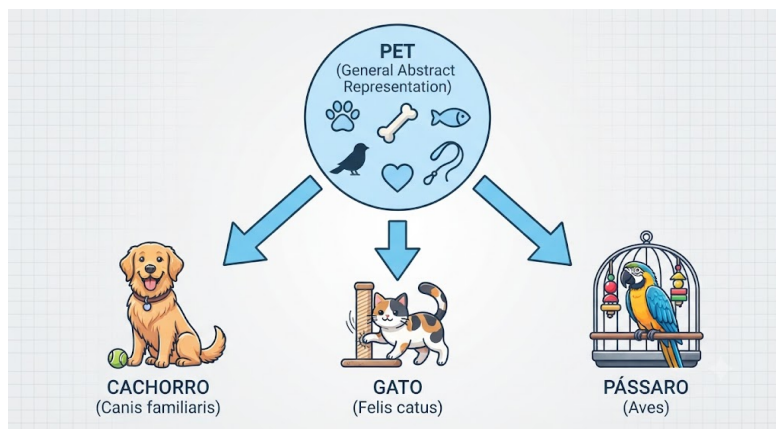


Figura 1.1: Abstração de pets para cadastrar derivados como cachorros, gatos e pássaros.  
Fonte: Imagem gerada por IA

Saber abstrair as informações que são importantes para um sistema é algo fundamental para a construção de software. Essa habilidade é adquirida com a prática e com o tempo.

## 1.2 Objeto

Uma das definições mais importantes é o objeto. Segundo Leite et al. (2016) *“Um objeto é a representação de um conceito/entidade do mundo real, que pode ser física (bola, carro, árvore etc.) ou conceitual (viagem, estoque, compra etc.) e possui um significado bem definido para um determinado software.”*

No exemplo anterior da clínica veterinária o objeto é o animal, assim como em um sistema de estoque o objeto pode ser o produto. Outro exemplo de objeto é uma viagem, que pode ter como atributos o destino, a data de ida e volta, o preço, entre outros.

Um objeto pode ser claramente definido através de:

- Um conjunto de atributos (características do objeto, o que ele é)
- Um conjunto de métodos (comportamento do objeto, o que ele faz)

### 1.2.1 Estado de um objeto e ciclo de vida

Depois que um objeto é criado, passa a ter um estado definido pelos valores de seus atributos. Este estado pode mudar ao longo do tempo conforme alterações feitas nos seus atributos pelos seus métodos. Um exemplo é um possível método `vacinar()` na **classe** `Pet` que altera o estado do atributo `vacinado` de `false` para `true`.

O ciclo de vida de um objeto é definido por três fases:

- Criação
- Uso
- Destruição

A criação ocorre quando o objeto é instanciado a partir de sua **classe**, o uso envolve a manipulação de seus atributos e métodos, e a destruição acontece quando o objeto deixa de ser necessário e é removido da memória pelo coletor de lixo do Java.

#### Observação

Apesar dos termos instância e objeto serem usados frequentemente como sinônimos, eles possuem significados distintos. **Objeto** é o conceito genérico de uma entidade criada a partir de uma classe, enquanto **instância** é o objeto concretizado (com valores), olhando especificamente para a relação dele com a sua classe de origem.

### 1.2.2 Identidade e igualdade de um objeto

A identidade de um objeto é única e imutável. Devido a isso, é muito difícil de comparar dois objetos, mesmo que eles possuam os mesmos atributos e valores. Para isso, o Java fornece o método `equals()` que compara se dois objetos são iguais, ou seja, se possuem os mesmos valores de atributos.

Esse e outros métodos serão tratados na seção (métodos e atributos estáticos) ??.

### 1.3 Classe

Um dos conceitos fundamentais da Programação Orientada a Objetos (POO) é a classe. Ela funciona como um molde ou modelo para a criação de objetos, representando a abstração de um conceito do mundo real. Na classe, definimos as características (**atributos**) e as ações (**métodos**) que os objetos desse tipo deverão possuir. Portanto, enquanto a classe é a definição generalizada, o objeto é a instância específica e concreta gerada a partir desse modelo.

Para nomear uma classe devemos utilizar **substantivos** e como uma boa prática deve iniciar com letra maiúscula. Em Java é comum criar um novo arquivo para cada classe do sistema. **Este arquivo deve ter o mesmo nome da classe.**

Veja o exemplo abaixo de uma classe baseada no exemplo da “pet”. Para replicar esse exemplo, crie um projeto Java como foi feito no capítulo 1 na seção??. Depois de criado o projeto, crie um arquivo novo do tipo *Java Class* na pasta **src**, de o nome de Pet, em seguida insira o código abaixo.

```
public class Pet {
    String nomeAnimal;
    int idadeAnimal;
    int sexoAnimal;
    boolean vacinado;
}
```

No exemplo anterior, definimos que as características pertinentes ao objeto Pet é o nome, a sua idade, seu sexo e o status de vacinação.

Agora na classe Principal onde está o método **main**, vamos criar o objeto e torna-lo concreto. Para isso basta utilizar a palavra chave **new**. Veja o exemplo abaixo:

```
public class Principal {
    public static void main(String[] args) {
        new Pet();
    }
}
```

O objeto é criado na memória, mas para acessá-lo, é necessário atribuí-lo a uma variável. Somente por meio de uma **referência** (uma variável nunca acessa o objeto diretamente na memória, sempre usamos a referência dele) podemos manipular esse objeto sem a necessidade de gerenciar a memória diretamente. Como vimos, o Java é uma linguagem fortemente tipada, o que significa que toda variável deve possuir um tipo definido. Portanto, para armazenar uma **instância** de “Pet”, devemos declarar uma variável do tipo “Pet”.

```
public class Principal {
    public static void main(String[] args) {
        Pet novoPet;
        novoPet = new Pet();
    }
}
```

```
}
```

Agora podemos adicionar valores às características do Pet, para utilizar os recursos (atributos, métodos, etc) de uma classe usamos a variável+“.”+recurso.

```
public class Principal {
    public static void main(String[] args) {
        Pet novoPet;
        novoPet = new Pet();
        novoPet.nomeAnimal = "Wando";
        novoPet.idadeAnimal = 1;
        novoPet.SexoAnimal = 1;
        novoPet.vacinado = false;
        System.out.println("O sexo do pet e: " +novoPet.sexoAnimal)
            ; //o sinal + em uma string significa ‘concatenar’
    }
}
```

Como vimos anteriormente, os métodos definem as ações que um objeto pode realizar ou o comportamento que ele pode exibir. Para ilustrar isso, vamos utilizar o atributo vacinado da classe Pet, esse atributo controlará o histórico de vacinação do animal. Além disso, implementaremos o método vacinar(), que será responsável por alterar esse estado de false para true, registrando a vacinação no sistema.

```
public class Pet {
    String nomeAnimal;
    int idadeAnimal;
    int sexoAnimal;
    boolean vacinado;
}
void vacinar() {
    this.vacinado = true;
    System.out.println(this.nomeAnimal + " foi vacinado com
        sucesso!");
}
}
```

O método vacinar() altera o status do objeto Pet para indicar que ele recebeu a dose necessária. Neste método, utilizamos a palavra-chave this para referenciar o atributo de instância vacinado da classe, diferenciando-o de qualquer parâmetro ou variável local que pudesse ter o mesmo nome. Essa prática é essencial para garantir que a alteração de estado seja aplicada corretamente ao objeto específico que está sendo vacinado. O arquivo principal deve ser estruturado da seguinte forma:

```
public class Principal {
    public static void main(String[] args) {
        // Instanciacao e manipulacao do objeto Pet
        Pet novoPet;
        novoPet = new Pet();
    }
}
```

## 1 Fundamentos de Orientação a Objetos

```
// Simulacao de uma acao de vaccinacao
// O metodo vacinar altera o estado interno do objeto
novoPet.vacinar();

// Exemplo de acesso e exibicao de informacoes
System.out.println("Status do pet vacinado: +" novoPet.
    vacinado);
}
```

## Referências Bibliográficas

Leite, T. et al. (2016). *Orientação a Objetos: Aprenda seus conceitos e suas aplicabilidades de forma efetiva*. Editora Casa do Código.